AI エージェントとマルチエージェントの歴史

知識工学から LLM、フィジカル AI、Agentic Robotics へ

知識を持つ AI から 知識を獲得し 協調し 行動する AI へ

Version 1.0（2026 年 11 月）

北陸先端科学技術大学院大学　西村研究室　ゼミ用資料

北陸先端科学技術大学院大学　博士前期課程 1 年

知能ロボティクス研究室 所属

武藤　雅裕

本稿の目的

本稿では、AI エージェントとマルチエージェントの発展を、単なる技術年表としてではなく、「知識をどのように獲得し、表現し、使い、共有し、行動を通じて更新していくか」という知識科学の観点から整理してみます。記号主義 AI、Blackboard、BDI、PRS、Contract Net、Ontology、Knowledge Graph を経て、LLM Agent、GraphRAG、Vision-Language-Action モデル、World Model、そして最近の Agentic Robotics や再帰的自己改善（RSI）へとつながる流れを追っていきます。

先に結論を書いておくと、LLM は古典的な知識工学を不要にしたわけではなく、知識獲得と形式化の「やり方」を変えたのだと考えています。いまの AI エージェントには、LLM の柔軟な生成能力に加えて、状態管理、計画、意味論、時間、因果、検証、実行可能性を支える外側の構造が、やはり必要になります。

Version 1.0 では、これまでの章に加えて、Agentic Robotics（第 12 章）、再帰的自己改善（第 13 章）、本稿でまだ足りていない視点（第 14 章）を新しく書き足しました。また、初めてこの分野に触れる人にもイメージがつかみやすいように、各章に図を 1 枚ずつ入れました。まだ荒削りなところも多いので、議論のたたき台として気軽にコメントをいただけるとうれしいです。

目次

序章　本稿の視点と問い
第 1 章　記号主義 AI と知識工学
第 2 章　Blackboard アーキテクチャ
第 3 章　BDI モデル
第 4 章　PRS と手続き的知識
第 5 章　Contract Net と分散協調
第 6 章　Ontology と Semantic Web
第 7 章　深層学習と強化学習
第 8 章　LLM エージェント
第 9 章　LLM マルチエージェント
第 10 章　GraphRAG
第 11 章　フィジカル AI
第 12 章　Agentic Robotics：考えて動くロボットへ（新規）
第 13 章　再帰的自己改善（RSI）（新規）
第 14 章　本稿でまだ足りていない視点（新規）
第 15 章　歴史の総括と研究課題
参考文献

序章　本稿の視点と問い

0.1 AI エージェントを知識の循環として捉える

AI エージェントとは、環境を観測し、内部状態を更新し、目標にもとづいて行動を選ぶシステムのことです。こう定義すると、いまの LLM エージェントはまったく新しい存在に見えるかもしれません。けれども、知識・目標・計画・行動・協調といった構成要素そのものは、古典的な AI の時代から繰り返し研究されてきました。

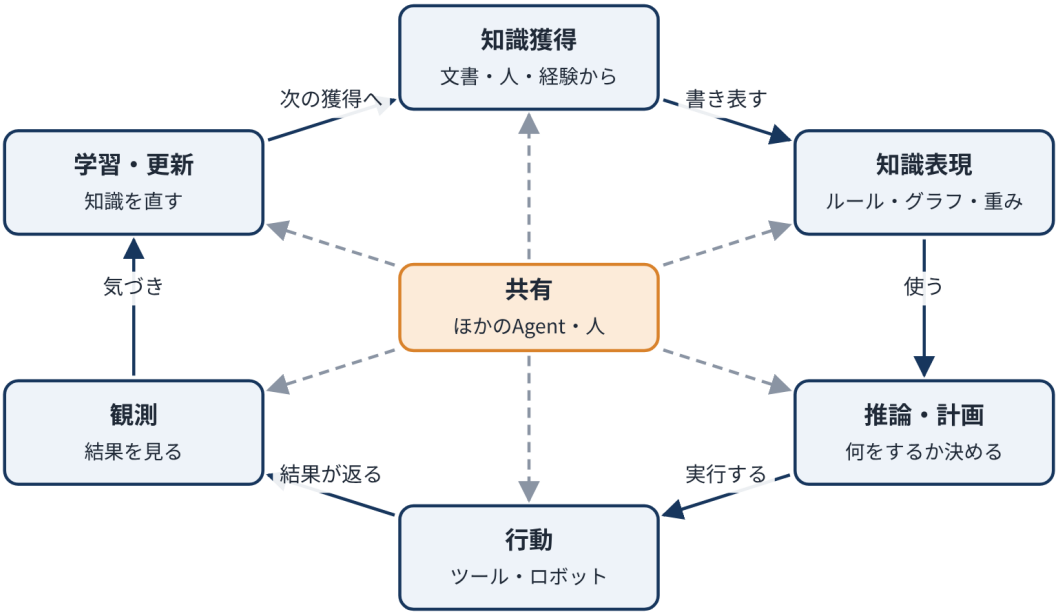


図 1　AI エージェントを「知識の循環」として見る。どの章の技術も、この輪のどこかを担当しています。

本稿では、各時代の技術を「知識獲得、知識表現、推論、計画、共有、行動、学習」の七つの観点から読んでいきます。新しい技術が古い技術を置き換えてきたというより、異なる知識処理のやり方が少しずつ積み重なり、いま改めて組み合わされつつある、という見方をとります。

0.2 二つの長期的なボトルネック

課題	内容	現在の変化	残る問題
知識獲得	専門家の知識や文書から、利用可能な知識を取り出す	LLM が候補を大量に抽出できる	暗黙知、誤情報、情報欠落、出典
形式化	自然言語の知識を規則、概念、関係、制約へ変換する	LLM が JSON、コード、グラフ、ルールを生成できる	意味整合性、例外、形式妥当性、保守

この二つのボトルネックは、LLM によってかなり緩和されましたが、なくなったわけではありません。人間が一から知識を書き下ろす工程から、LLM が生成した知識の候補を人間や形式的な手法で確かめる工程へと、作業の重心が移ってきている、と捉えるのがよさそうです。

第 1 章　記号主義 AI と知識工学

1.1 知能を記号操作として実装する

初期の人工知能研究では、世界についての事実や規則を記号で表し、推論規則を使って結論を導く方法が中心でした。エキスパートシステムは、専門家の判断を if-then ルールとして書き下ろし、推論エンジンがそれを個々の事例に当てはめることで、診断や助言を行っていました。

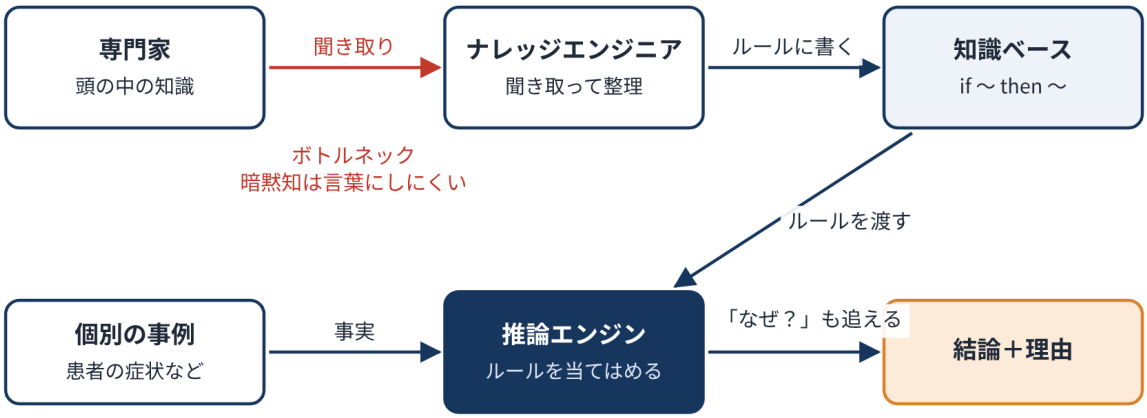


図 2　エキスパートシステムのしくみ。赤い矢印（専門家からの聞き取り）が「知識獲得のボトルネック」です。

この方式のよいところは、知識と推論の過程を目に見える形で示せることです。なぜその結論になったのかを規則の連鎖としてたどれますし、知識の追加や修正も原理的には可能でした。その一方で、現実の専門知識を漏れなく、しかも矛盾なく書き下ろすのはとても難しいことでした。

1.2 Knowledge Level

Newell は、知的システムを実装部品のレベルだけでなく、「何を知り、どんな目標を持ち、その知識を合理的にどう使うか」という Knowledge Level から捉えました [1]。この視点は、いまの AI エージェントを考えるうえでも役に立ちます。モデル、メモリ、ツール、ワークフローを分けて実装したとしても、エージェント全体として使う知識と目標がかみ合っていなければ、一貫した行動にはなりません。

1.3 Knowledge Acquisition Bottleneck

知識工学では、専門家が持っている知識をナレッジエンジニアが聞き取り、概念・規則・例外として実装する工程が欠かせませんでした。この工程には時間も費用もかかりますし、専門家自身も「なんとなくこう判断している」という暗黙の基準をうまく言葉にできないことがよくあります。これが、知識獲得のボトルネックと呼ばれる問題です。

さらに、ルールが増えるほどルール同士の関係は複雑になり、例外処理や保守が大変になっていきます。知識が正しいかどうかだけでなく、どんな状況で有効なのか、誰が承認したのか、いつ

変更されたのかも管理しなければなりません。後に登場する Ontology、Temporal Knowledge Graph、Agent Memory の研究は、この問題を形を変えて引き継いでいると言えます。

第 2 章　Blackboard アーキテクチャ

2.1 複数の知識源を協調させる

HEARSAY-II は、音声理解という不確実で複雑な問題に取り組むために、複数の専門的な知識源が共有の作業領域である Blackboard を介して協調する構造を採用しました [2]。音響、音素、単語、構文といった異なるレベルの仮説を同じ場所に書き込み、ほかの知識源がその状態を見ながら処理を進めていきます。

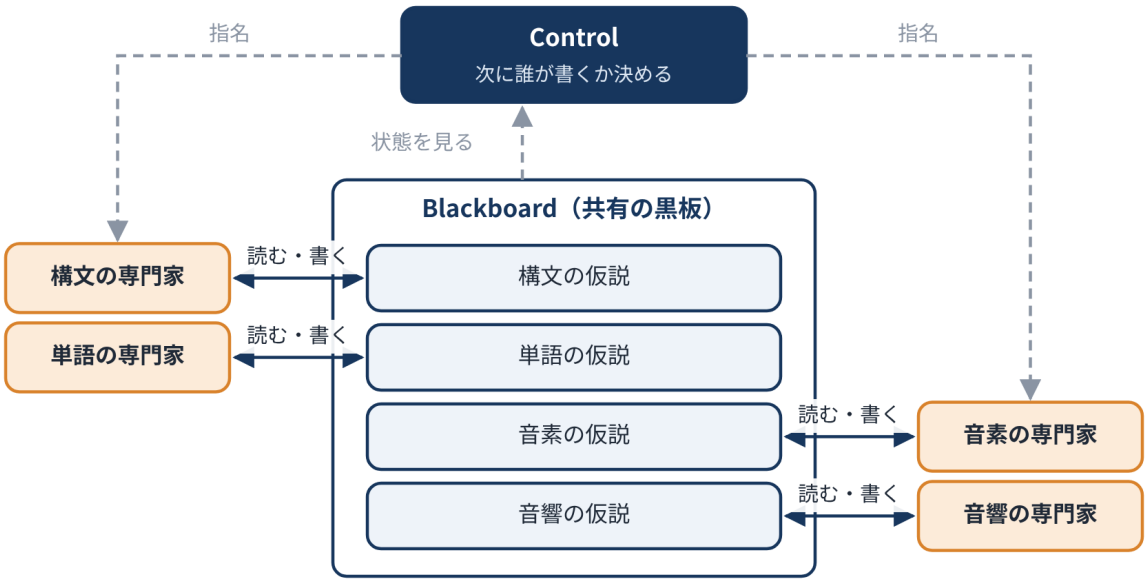


図 3　HEARSAY-II の Blackboard。専門家どうしは直接話さず、黒板の途中結果だけを通じて協力します。

ポイントは、すべての能力を一つの巨大な推論器に詰め込まなかったことです。各知識源は自分が扱える部分問題に集中し、Blackboard 上の途中結果を通じてゆるやかにつながります。この考え方は、いまの共有メモリ型マルチエージェントやイベント駆動のワークフローと比べてみると、よく似ていて面白いところです。

2.2 現代の共有状態との対応

Blackboard の要素	役割	現代の実装例
Blackboard	仮説、部分結果、状態を共有	State Store、Task DB、Knowledge Graph
Knowledge Source	特定領域の処理を実行	専門 Agent、Tool、Skill、Microservice
Control	次に実行する知識源を選ぶ	Supervisor、Scheduler、Workflow Engine
Trigger	状態変化に応じて処理を起動	Event、Queue、Rule、Condition

LLM マルチエージェントで共有のチャット履歴だけを Blackboard 代わりにすると、「状態」と「会話」が混ざりやすくなります。タスクの状態、事実、仮説、判断の根拠、実行履歴を型付きで分けておくことが、古典的な Blackboard の知見をいまに生かす一つの方法です。

第 3 章　BDI モデル

3.1 Belief Desire Intention

BDI は、エージェントの内部状態を Belief（信念）、Desire（願望）、Intention（意図）の三つで捉えるモデルです。Belief は世界についてエージェントが持っている情報、Desire は実現したい状態の候補、Intention は実際に取り組むと決めて持ち続ける目標や計画を指します。Bratman の実践的推論の議論が、その哲学的な背景になっています [6]。

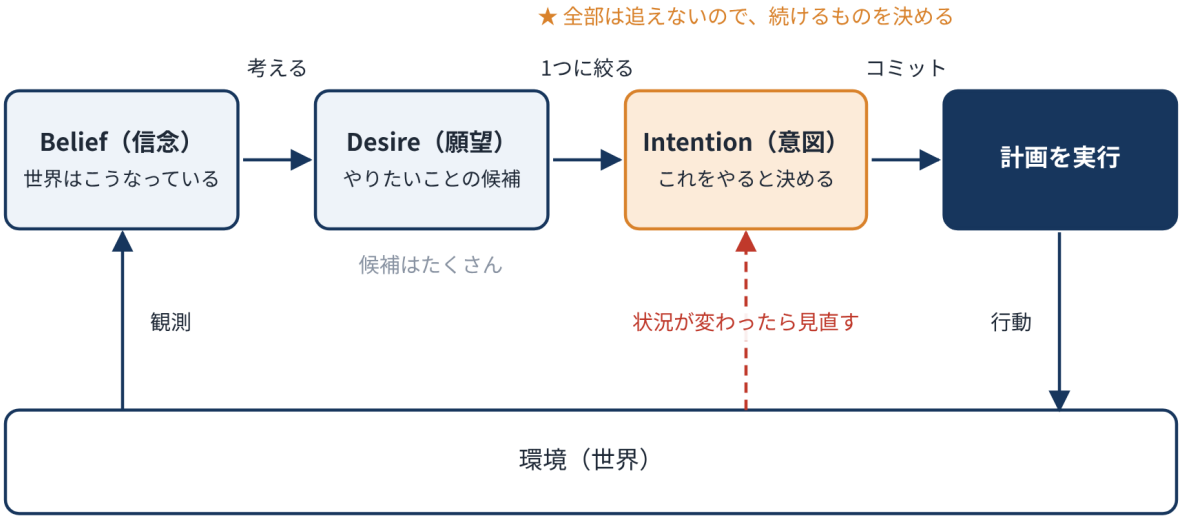


図 4　BDI の意思決定ループ。オレンジの Intention が「何にコミットするか」で、赤い点線が再検討です。

Desire と Intention を分けているところが、大事なポイントです。エージェントは望ましい状態をたくさん持てますが、そのすべてを同時に追いかけることはできません。Intention は、限られた資源のなかで「何にコミットするか」を表し、状況が変わってもある程度一貫した行動を保つ役割を果たします。

3.2 BDI の意思決定ループ

1. 環境を観測して、Belief を更新します。
2. Belief と目標から、実行できそうな選択肢を挙げます。
3. 制約・優先度・資源を考えて、Intention を選びます。
4. Intention に対応する計画を実行します。
5. 成功・失敗・環境の変化に応じて、計画を見直します。

Rao と Georgeff は、BDI を理論から実装へつなぐ枠組みとして整理しました [8]。さらに Rao の AgentSpeak(L)は、BDI エージェントをプログラミング言語として書ける形にしています [9]。いまの LLM エージェントでも、ユーザーの依頼をそのままプロンプトに入れるだけでは、Belief・Goal・Intention の区別があいまいになりがちです。これらを外部の状態としてはっきり管理しておくと、長期タスクの一貫性や、途中からの再開しやすさ、あとから確かめられる監査性を高められます。

3.3 LLM エージェントへの示唆

BDI 概念	LLM エージェントでの対応	設計上の注意
Belief	会話履歴、RAG、外部メモリ、KG	事実、推測、期限、出典を区別する
Desire	ユーザー要求、候補目標	相反する目標と優先度を管理する
Intention	実行中の計画、タスクキュー	コミットメントと中断条件を明示する
Reconsideration	Reflection、再計画	毎回計画を破棄せず、変更条件を定める

第 4 章　PRS と手続き的知識

4.1 宣言的知識から行動手順へ

Procedural Reasoning System（PRS）は、目標を達成するための手続き的知識を Plan や Knowledge Area として持ち、そのときの Belief と Goal に合わせて適切な計画を選んで実行するアーキテクチャです [4]。単なるルール推論ではなく、行動の途中の状態や、途中で起きるイベントも扱えるのが特徴です。

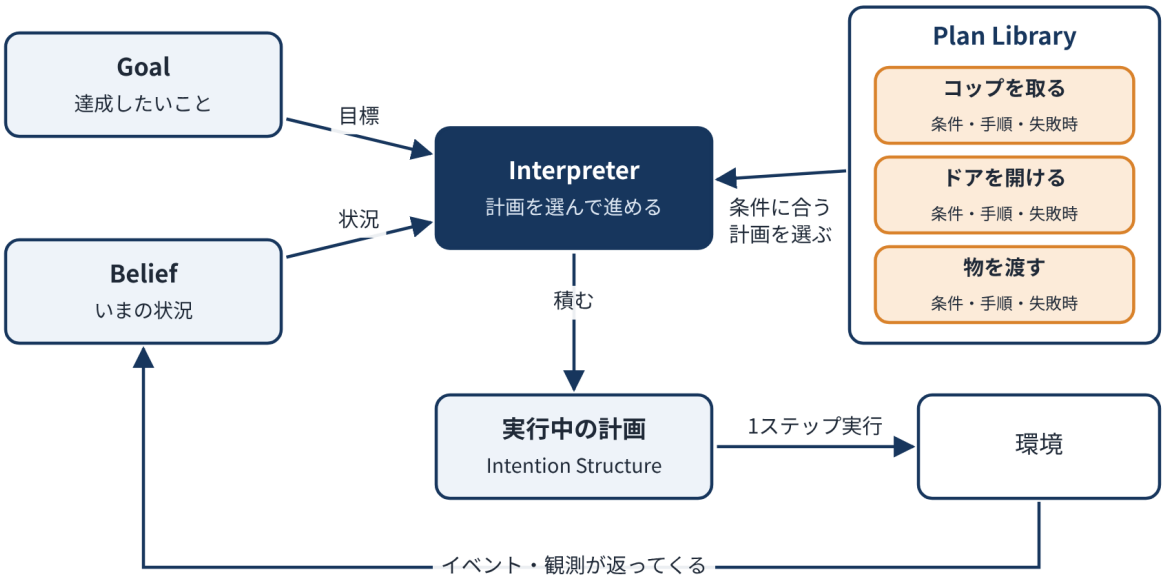


図 5　PRS のしくみ。あらかじめ用意した手順書（Plan Library）から、状況に合うものを選んで実行します。

ここでいう計画は、決まった手順を一列に並べたものとは限りません。適用条件、達成すべき目標、実行ステップ、分岐、失敗したときの処理まで含んでいます。この構造は、いまの Agent Skill、Workflow、Tool Sequence、Runbook とよく似ています。

4.2 PRS と現在の Agent Harness

PRS の構成	意味	現代の実装
Belief Database	環境と内部状態	State、Memory、RAG、KG
Goals	達成すべき状態	Task Spec、Acceptance Criteria
Plan Library	再利用可能な行動手順	Skill、Workflow、Playbook
Intention Structure	実行中の計画と階層	Task Graph、Checkpoint
Interpreter	状況に応じて計画を進める	Orchestrator、Agent Runtime

LLM は柔軟に計画を作るのが得意な一方で、同じ条件でも毎回ちがう手順を出してくることがあります。リスクの高い業務では、承認済みの手続き的知識を Plan Library として持っておき、LLM

には計画の選択、引数の補完、例外が起きたときの候補づくりを任せる、という役割分担が向いていると考えられます。

第 5 章　Contract Net と分散協調

5.1 タスク割り当てを交渉する

Contract Net Protocol は、分散問題解決においてタスクを動的に割り当てるためのプロトコルです [3]。Manager がタスクを公告し、候補となる Contractor が自分の能力・費用・期限などを含めた提案を返し、Manager がその中から採択します。担当をあらかじめ固定せず、そのときの状況に応じて決めるのが特徴です。

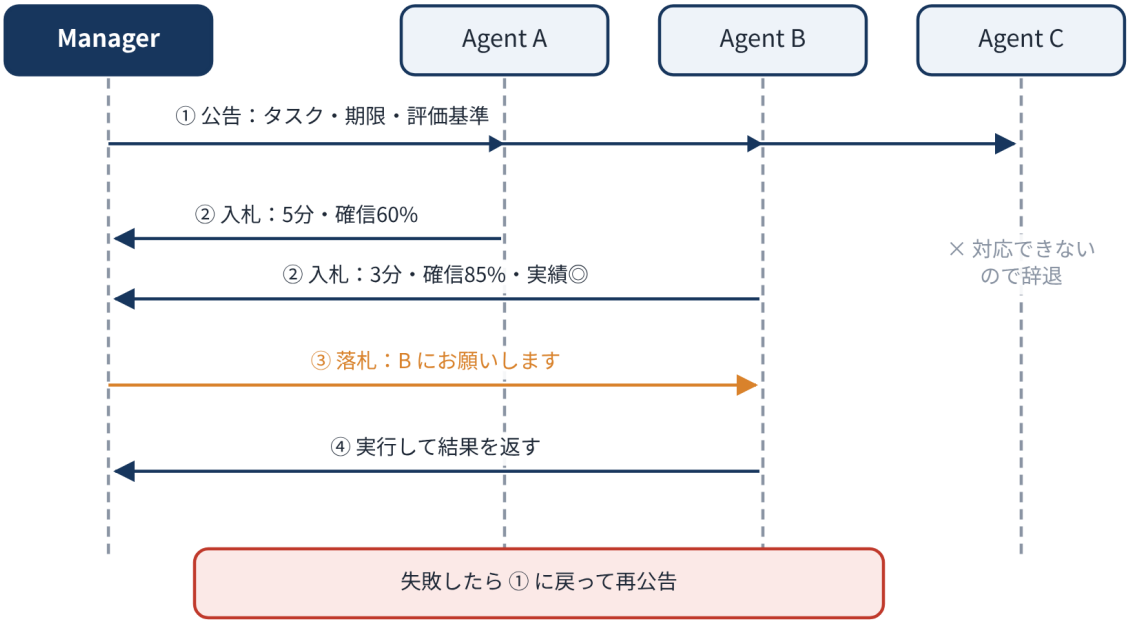


図 6　Contract Net の流れ。担当を最初から決めず、「入札」で一番合う相手を選びます。

1. Manager が、タスクの内容・制約・期限・評価基準を公告します。
2. 候補の Agent が、能力・見積もり・確信度・必要な資源を提案します。
3. Manager が提案を比べて、契約を与えます。
4. Contractor が実行し、進み具合と結果を返します。
5. 失敗や条件の変化があれば、再公告や再割り当てを行います。

5.2 LLM マルチエージェントへの適用

LLM エージェントでは、Supervisor が Agent の名前だけを見て担当を決めると、能力の重複や割り当てミスが起きやすくなります。Contract Net の考え方を取り入れて、各 Agent が使える Tool、必要なコスト、予想時間、確信度、前提条件を構造化して返すようにすれば、割り当てを評価できる形にできます。

ただし、LLM が自分で申告する確信度は、きちんと校正されていないこともあります。過去の成功率、検証結果、実行コストといった実際の観測値と組み合わせて入札を評価する必要があります。会話のうまさよりも、協調のプロトコルのほうが大事だということを示す例だと思います。

第 6 章　Ontology と Semantic Web

6.1 概念化を共有する

Gruber は Ontology を「共有された概念化の明示的な仕様」として論じました [7]。Ontology は単なる用語集ではなく、クラス・関係・属性・制約を使って、対象領域をどう捉えるかをはっきりと書き表したものです。異なるシステムや組織が同じ言葉をちがう意味で使ってしまう問題に対応するための道具でもあります。

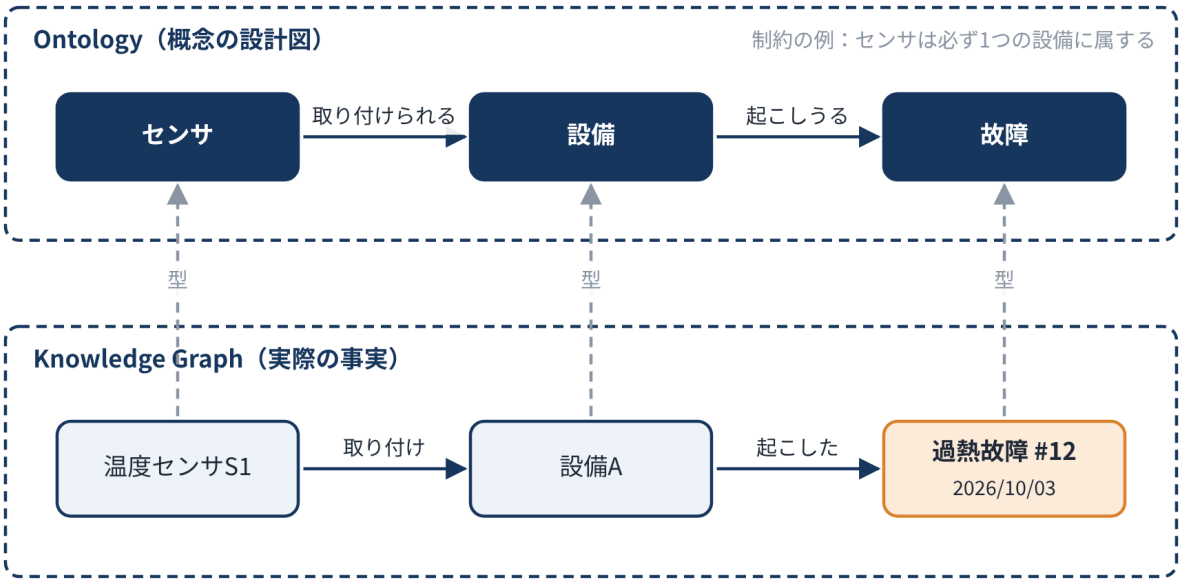


図 7　Ontology は「設計図」、Knowledge Graph は「設計図に沿って書いた実際の事実」。点線は「この事実はどの型か」を表します。

Semantic Web は、Web 上のデータに機械が読める意味を与え、ソフトウェアエージェントがさまざまな情報源を統合して使えるようにする構想でした [10]。RDF、RDFS、OWL といった標準は、識別子・関係・型・論理的な制約を共有するための基盤になりました [11]。

6.2 Ontology Knowledge Graph Semantic Layer

層	主な役割	例
Ontology	概念、関係、制約の定義	設備、部品、故障、作業の型と制約
Knowledge Graph	個別の実体、関係、事実を保持	設備 A にセンサ S が接続されている
Semantic Layer	利用者や Agent に統一された意味を提供	同一 KPI、業務語彙、データ契約
Context Layer	タスクに必要な情報を動的に構成	役割、時点、権限に応じた文脈

いまの AI エージェントで考えると、Ontology は厳密な意味と制約、Knowledge Graph は実世界の個別の知識、Semantic Layer は業務上の意味の一貫性、Context Layer はいまのタスクに必要な情報の切り出しを担う、と整理できます。これらは同じものではなく、お互いを補い合う関係にあります。

第 7 章　深層学習と強化学習

7.1 知識を記述する方式から学習する方式へ

深層学習の発展によって、画像・音声・言語の特徴を人手で規則にするのではなく、大量のデータから表現を学習する方法が主流になりました。知識は、明示的な記号や規則だけでなく、ニューラルネットワークのパラメータや潜在表現のなかにも分散して保持されるようになったわけです。

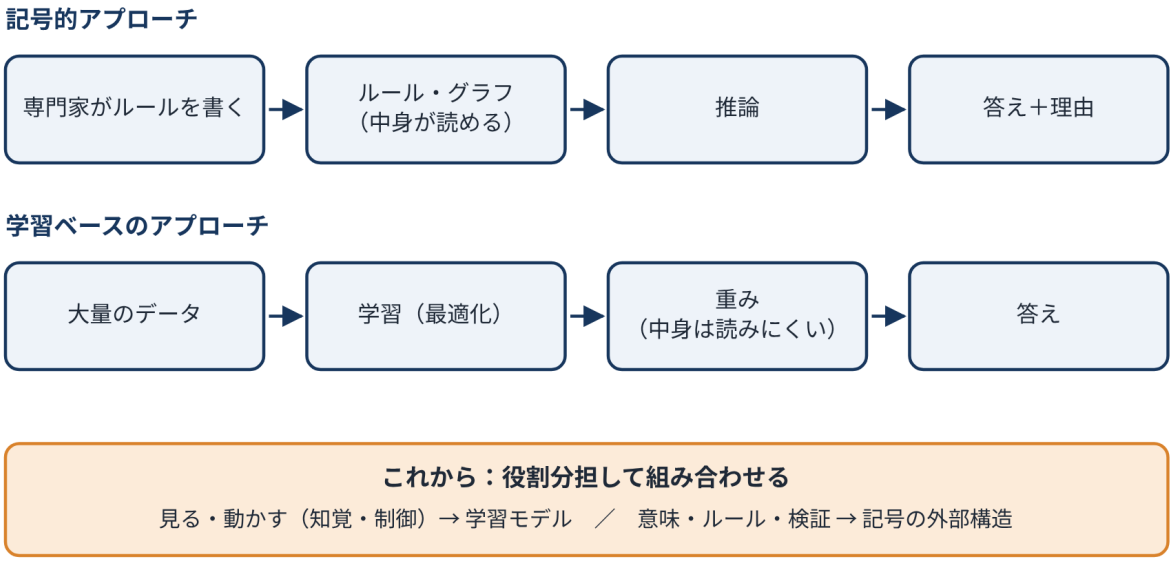


図 8　知識の「書き方」の違い。上は人が書いて中身が読める、下はデータから学んで強いが中身が読みにくい。

強化学習では、エージェントが環境とやり取りしながら、累積報酬が最大になるような方策を学びます。Deep Q-Network は深層学習と強化学習を組み合わせ、画像入力から行動価値を学習しました [12]。AlphaGo は、方策ネットワーク、価値ネットワーク、探索を組み合わせ、学習された知識と明示的な探索をうまく両立できることを示しました [13]。

7.2 記号的知識と学習された知識

観点	記号的アプローチ	学習ベースのアプローチ
知識表現	ルール、論理、グラフ	重み、埋め込み、潜在状態
獲得方法	専門家による定義	データと最適化
強み	意味、制約、説明、検証	知覚、一般化、複雑なパターン
弱み	構築と保守のコスト	説明、保証、分布外への対応
統合の方向	Neuro-symbolic、KG 補助、制約付き学習	外部メモリ、Tool、World Model

いま問われているのは、どちらか一方を選ぶことではありません。連続的な知覚や制御は学習モデルに任せ、意味・制約・履歴・検証は記号的な外部構造に任せる、という役割分担が大切になってきています。

第 8 章　LLM エージェント

8.1 LLM が知識獲得と形式化を変える

これまでは、専門家の説明をナレッジエンジニアが読み解き、ルールや知識ベースに変換する必要がありました。いまは、LLM を使って文書から概念や関係を取り出し、Knowledge Graph や Rule の候補を作ることができます。さらに、人間が自然言語で伝えた目標から、行動計画やプログラムを生成することもできるようになりました。

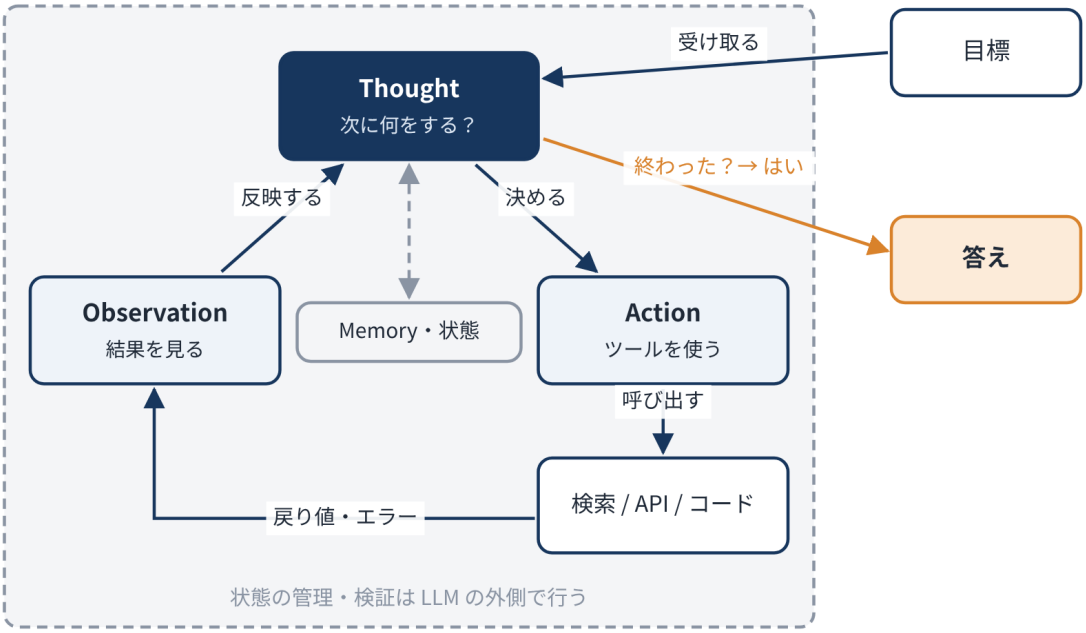


図 9　ReAct の「考える → 動く → 見る」ループ。点線の枠（状態管理・検証）は LLM の外で用意する必要があります。

従来の課題	LLM による変化	残されている課題
知識獲得のボトルネック	文書や会話から知識候補を抽出できる	暗黙知、誤情報、情報の欠落、出典
形式化のコスト	自然言語から Rule、JSON、Graph、Code を生成できる	意味の整合性、形式的妥当性、例外処理

LLMs4OL は、LLM による用語の型付け、概念階層の発見、関係抽出を評価し、LLM が Ontology Learning を支援できる可能性がある一方で、高度な推論や専門知識が必要な作業ではまだ十分ではないことを示しました [18]。LLM は知識工学を不要にしたのではなく、生成された知識の候補を人間と形式的手法で確かめる工程へと、仕事の形を変えたのだと言えます。

8.2 ReAct 推論と行動を交互に実行する

ReAct は、言語モデルによる Reasoning（推論）と外部環境への Action（行動）を交互に行う仕組みを提案しました [15]。Tool から得た Observation（観測）を次の推論や行動に反映させることで、モデルの中にない情報や計算結果を使えるようになります。

1. ユーザーの目標を受け取ります。
2. 次に必要な情報や行動を考えます。
3. DB 検索、API 呼び出し、コード実行などを行います。
4. 戻り値やエラーを観測します。
5. 結果を反映し、終了条件を満たすまで次の行動を決めていきます。

ReAct はあくまで推論と行動の実行パターンであり、Belief・Desire・Intention を厳密に区別する BDI アーキテクチャとは異なります。目標へのコミットメント、途中の状態、失敗したときの再計画、すでに実行した行動をきちんと管理するには、LLM の外側に状態管理と検証の仕組みが必要になります。

8.3 現代の LLM エージェントの内部構造

古典的概念	現代の実装例
Belief	会話履歴、外部 Memory、RAG、Knowledge Graph
Desire / Goal	ユーザー指示、Task Definition、Goal State
Intention	実行中の Plan、Task Queue、Workflow State
Procedural Knowledge	Tool Definition、Code、Skill、Runbook
Reasoning / Planning	Task Decomposition、Plan Generation、Replanning
Action	API Call、Code Execution、Robot Command
Observation	Tool Result、Error、Sensor Data
Reflection	実行結果の評価、失敗分析、次の試行への反映

Reflexion は、行動結果に対する言葉によるフィードバックを Memory に残し、次の試行に生かす仕組みを提案しました [22]。知識科学の観点からは、Memory を単なるテキストの履歴ではなく、出典・確信度・有効期間・関係を持った「確かめられる知識」として構造化していくことが課題になります。

第 9 章　LLM マルチエージェント

9.1 専門性を持つ Agent を組み合わせる

LLM の登場によって、自然言語で役割や目標を指定した Agent を手軽に作れるようになりました。AutoGen は、LLM・人間・Tool を組み合わせ、複数の Agent が会話を通じてタスクを進める枠組みを提案しました [16]。

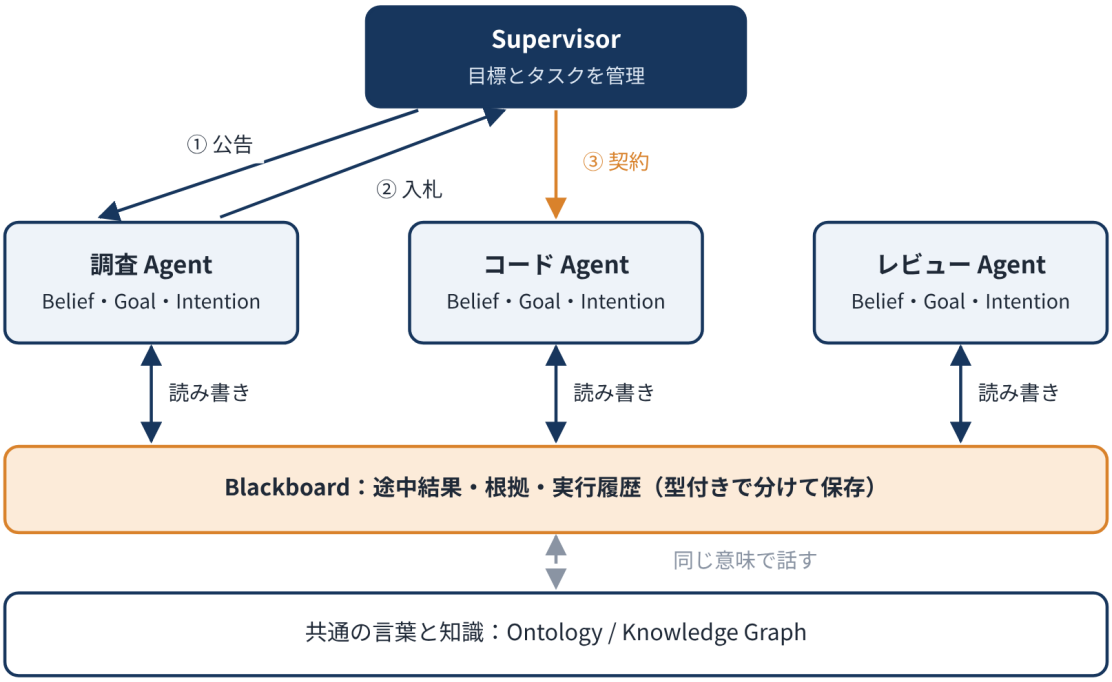


図 10　第 9.3 節の統合案を一枚に。上から「割り当て（Contract Net）→ 各 Agent（BDI）→ 共有（Blackboard）→ 共通の意味（Ontology/KG）」。

Generative Agents は、経験を Memory に蓄え、そこから Reflection と Planning を生み出す Agent によって、人々の社会的なやり取りをシミュレーションしました [17]。ざっくり言えば、AutoGen はタスクの協調を、Generative Agents は Memory と Planning がふるまいにつながっていく過程を、主に扱っています。

9.2 マルチエージェントアーキテクチャの分類

Architecture	仕組み	古典研究との関係
Supervisor	中央 Agent がタスクを分解し割り当てる	中央集権的な分散問題解決
Planner Executor	計画と実行を別の Agent に分ける	PRS の計画と実行の分離
Blackboard	複数 Agent が共有状態を読み書きする	HEARSAY-II
Contract Net	候補 Agent が提案し担当を交渉する	Contract Net Protocol
Peer to Peer	Agent が対等に通信する	分散 Multi-Agent System
Debate Review	提案を相互批評し検証する	分散協調と合意形成

自然言語でやり取りできるからといって、うまく協調できるとは限りません。タスクの重複、矛盾、無限ループ、誤った情報の共有などが起こりうるため、タスク割り当て、協調プロトコル、知識共有、組織構造といった仕組みが必要になります。

9.3 Blackboard Contract Net BDI の統合案

ここで考えている統合案では、Supervisor が目標とタスクを管理し、Contract Net が能力とコストにもとづいてタスクを割り当てます。各 Agent は Belief・Goal・Intention を持ち、Blackboard が状態と根拠を共有し、Ontology と Knowledge Graph が共通の意味を支える、という構成です。

構成要素	担当する課題
BDI	各 Agent の内部状態、目標、コミットメント、再検討
Contract Net	能力、費用、期限に基づく動的なタスク割り当て
Blackboard	途中状態、部分結果、根拠、履歴の共有
Ontology / KG	共通語彙、関係、制約、知識の来歴と時間

第 10 章　GraphRAG

10.1 LLM 内部の知識だけでは十分ではない

LLM は大量のデータから学習していますが、企業固有の設備情報や最新の業務状況、担当者どうしの関係などを、いつも正確に把握しているわけではありません。また、長い文書を Context に入れても、関連する情報がどこにあるかによって性能が変わってしまうことが、Lost in the Middle として報告されています [23]。

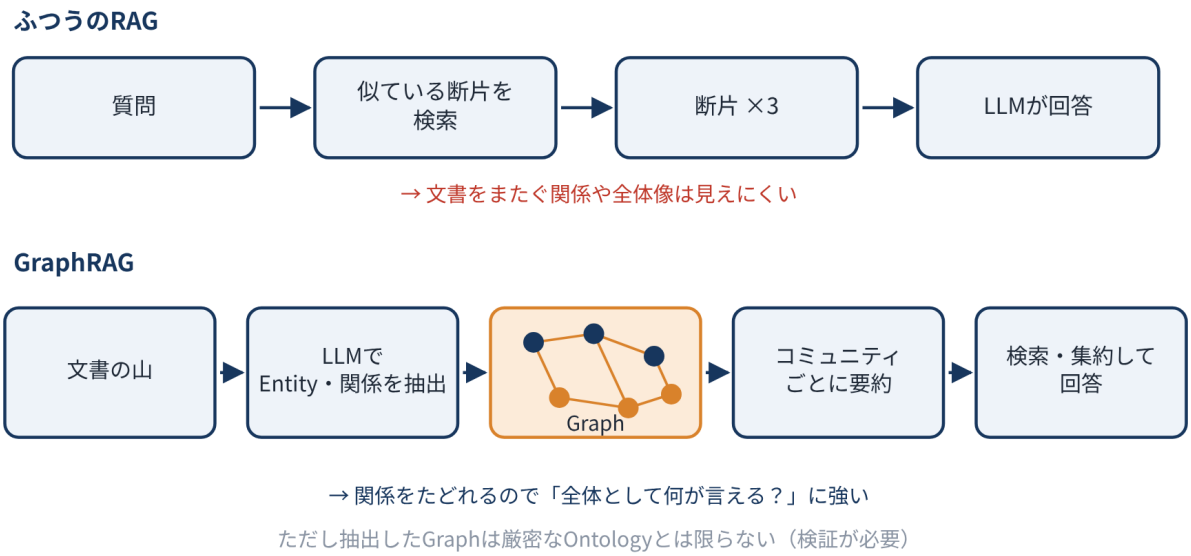


図 11　ふつうの RAG と GraphRAG の違い。GraphRAG は先に「関係の地図」を作ってから質問に答えます。

外部の知識を検索して LLM に渡す RAG は、この問題への基本的な対策です。ただ、単純な Vector Search だけでは、複数の文書にまたがる関係や、組織全体の構造、全体のテーマの要約などは扱いにくいことがあります。そこで、Knowledge Graph を検索と情報の集約に使う GraphRAG が注目されるようになりました。

10.2 GraphRAG が変えたこと

Microsoft Research の GraphRAG は、文書から Entity と Relation の Graph を作り、Community ごとの要約を生成することで、文書集合全体についての質問に答えられるようにする方法を提案しました [20]。処理の流れは、文書の入力、知識の抽出、Graph の構築、Community の検出と要約、質問に応じた検索、回答の生成、と整理できます。

大事なのは、LLM が文書から Graph を作ることで、人手で行っていた知識の構造化の一部を自動化できる点です。ただし、こうして抽出された Entity Graph は、必ずしも形式的な Ontology にもとづく Knowledge Graph ではありません。概念の厳密な定義、Cardinality、排他性、時間の制約などは、別に設計して確かめる必要があります。

10.3 知識科学から見た次の課題

たとえば、工場の設備管理 Agent が設備 A の温度上昇を分析する場面を考えてみます。関連する Sensor、Part、Failure、Inspection History は Graph から取り出せます。しかし、「部品 B が原因だ」と結論づけるには、時間的な前後関係、物理的な因果、Sensor の信頼性、ほかの仮説、介入した結果なども考える必要があります。

技術	AI エージェントへの役割
Ontology	設備、故障、作業などの概念と制約を定義する
Knowledge Graph	設備、部品、Sensor、作業者の関係を管理する
Temporal KG	いつ、どの関係や状態が成立していたかを保持する
Causal Model	観測相関と因果を区別し、介入効果を検討する
GraphRAG	質問やタスクに必要な関係情報を取得し集約する

LLM は知識の抽出と自然言語での利用を柔軟にし、Ontology と Knowledge Graph は意味・制約・履歴をはっきりさせて確かめられるようにします。そこに Temporal KG と Causal Model を組み合わせ、「いま有効な知識」と「因果についての仮説」を区別できるようにすることが、次の研究課題になると考えています。

第 11 章　フィジカル AI

11.1 言語理解から物理世界の行動へ

LLM にコップを持ち上げる手順を説明させることはできます。でも、実際の Robot がそれを実行するには、位置、近づく方向、つかむ力、衝突、物体の状態、失敗したときの修正まで扱わなければなりません。自然言語で書かれた知識と、物理世界で実行できる行動とのあいだには、まだ大きな隔たりがあります。

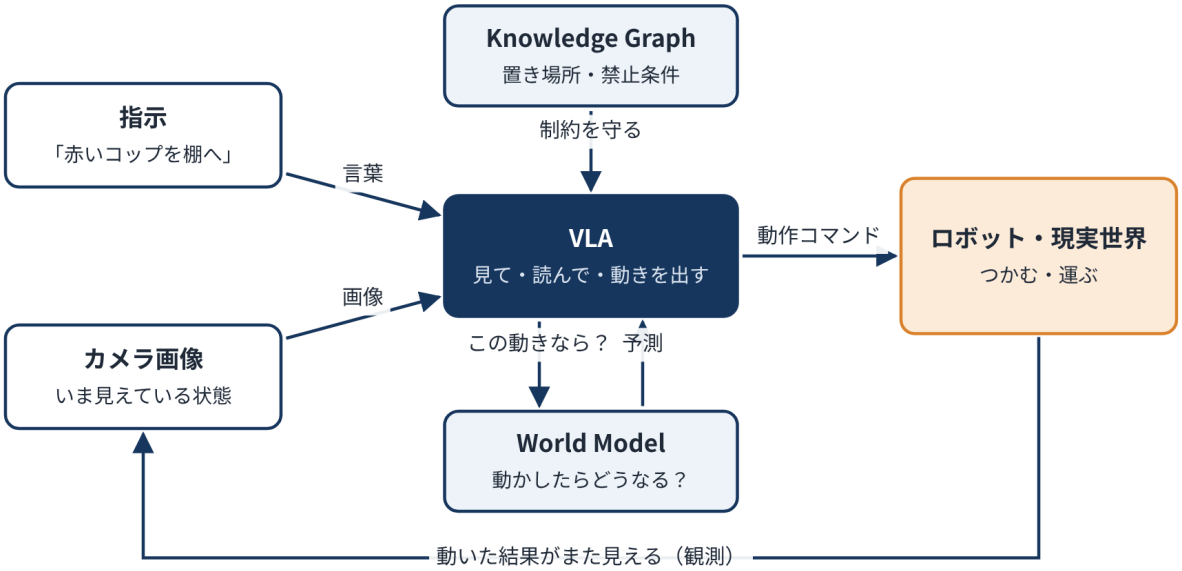


図 12　「赤いコップを棚へ」の例。言葉の知識（KG）と予測（World Model）と行動（VLA）が組み合わさる場面です。

SayCan は、LLM が持つ意味的な知識と、Robot Skill の実行可能性を組み合わせて、自然言語の指示を現実の Robot Action へとつなげました [14]。知識科学の観点から見ると、言葉で表された知識を、いまの環境で実行できる行為へと Grounding（接地）する問題だと言えます。

11.2 Vision Language Action モデル

RT-2 は、画像と言語に加えて Robot Action も扱う Vision-Language-Action（VLA）モデルを提案し、Web 由来の視覚・言語の知識を Robot Control へ転移させることを試みました [19]。OpenVLA は複数の Robot の Demonstration Data を使ったオープンソースの VLA を示し [21]、π0 は事前学習済みの VLM と Flow Matching を組み合わせて Action を生成しています [24]。

この流れのなかで、人間がすべての行動規則を書き下ろす必要性は小さくなってきました。その一方で、学習された Policy がなぜその行動を選んだのか、制約を守れているのか、失敗の兆しをどう見つけるのか、といった点はまだ課題として残っています。

11.3 World Model

World Model は、環境の状態とその時間的な変化を内部表現として学習し、Action によって将来の状態がどう変わるかを予測するモデルです。いまの状態を認識するだけでなく、いくつかの行動候補の結果を内部で比べられる点が重要です。

BDI の Belief は、エージェントが世界について持っている情報を表します。一方、World Model は、いまの状態に加えて Dynamics（変化の仕方）と Action Effect（行動の効果）を予測します。両者は同じものではありませんが、「エージェントが何を知り、どんな未来を予測して行動するのか」という共通の研究テーマを持っています。

11.4 Knowledge Graph と World Model

観点	Knowledge Graph	World Model
主な表現	Entity、Relation、Attribute	State、Latent Representation、Transition
主な目的	知識の表現、検索、共有、制約	環境変化と行動結果の予測
時間	Temporal KG として明示	Dynamic Transition として学習
強み	意味、関係、根拠を管理	複雑な変化を Data から学習
主な課題	知識獲得と Grounding	予測精度、説明、安全性

たとえば、赤いコップを棚へ移す場面を考えてみます。Knowledge Graph は対象物、置き場所、使ってよい道具、禁止条件を表し、World Model はつかんだり触れたりしたときの状態の変化を予測し、VLA は Observation と Instruction から Action を生成します。これらを組み合わせることで、記号的な知識と身体的な行動をつなげられる可能性があります。

ただし、Graph 上の概念と連続的な Sensor Data をどう対応づけるのか、予測結果をどの制約で確かめるのか、失敗した経験をどのくらいの粒度で Knowledge に戻すのか、といった点はまだ解決されていません。触覚を含む World Model では、接触の前後の潜在状態と、明示的な Failure Concept を結びつけることが研究課題になると考えています。この「考える部分」と「動く部分」をどうつなぐかは、次の第 12 章でもう少し詳しく見ていきます。

第 12 章　Agentic Robotics：考えて動くロボットへ

12.1 Agentic Robotics とは

2025 年ごろから、LLM エージェントで使われてきた考え方をロボットに持ち込む「Agentic Robotics」という言葉をよく目にするようになりました。CoRL 2026 の Agentic Robotics ワークショップでは、タスク分解、コード生成、ツール呼び出し、記憶の管理、途中のフィードバックにもとづく計画の見直しといったエージェント的な手法を、学習済みの Policy の上で推論時に組み合わせる研究として整理されています [29]。

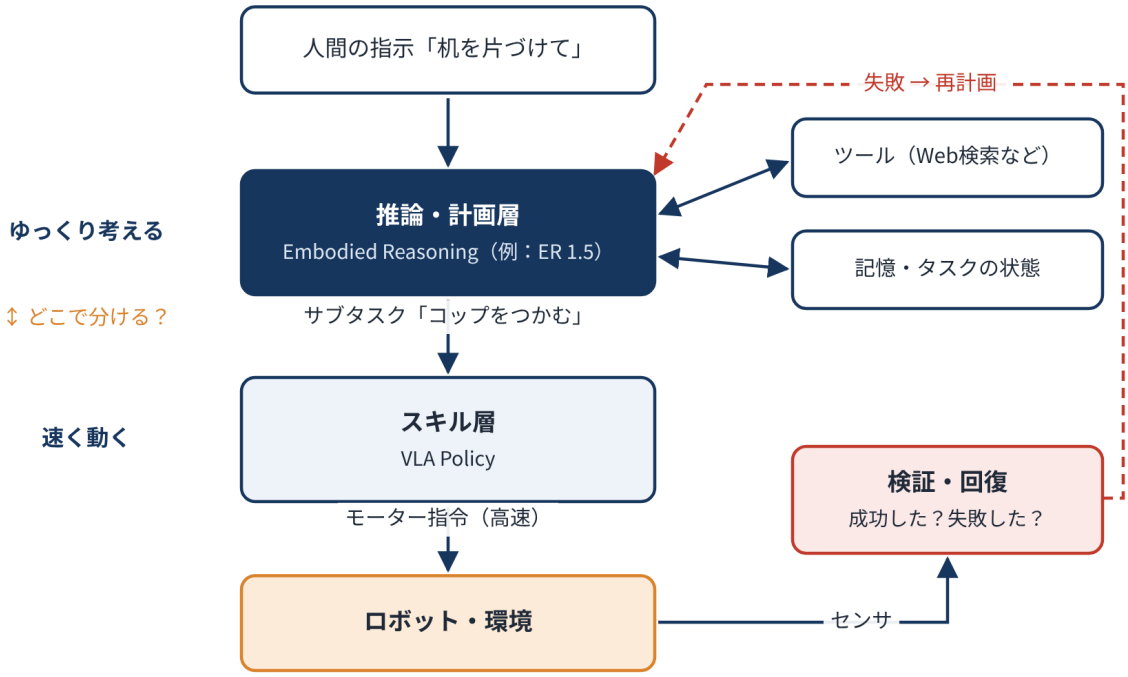


図 13　Agentic Robotics の二層構造。「考える層」と「動く層」をどこで分け、どうつなぐかが研究の焦点です。

デジタル世界のエージェントとの大きなちがいは、物理世界ならではの難しさです。ロボットの行動は連続的で、しばしば取り消しがききません。観測にはノイズがあり、見えていない部分も多く、しかもリアルタイムで判断しなければなりません [29]。第 11 章の VLA が「一つのモデルで見て動く」方向だとすれば、Agentic Robotics は「考える部分と動く部分をどう組み合わせるか」に重心を置いた流れだと言えます。

12.2 考える Agent と動く Policy の二層構造

初期の例としては、LLM にロボット制御のコードを書かせる Code as Policies [25] や、第 11 章で触れた SayCan [14] があります。最近ではこれが、推論を担うモデルと VLA を組み合わせた階層構造へと発展しています。Gemini Robotics 1.5 では、Embodied Reasoning モデル（Gemini Robotics-ER 1.5）が空間の理解、タスクの計画、進み具合の推定を担い、VLA である Gemini Robotics 1.5 が実際の動作を生成します。VLA 側も、動く前に自然言語で考える「Thinking」の仕組みを持ち、異なるロボットの身体から集めたデータで学べる Motion Transfer を取り入れています [26]。

Hu らは、こうした階層型 VLA エージェント（Hi-VLA）の設計を一つの枠組みで比較し、どのモデルを使うかと、層と層のあいだのインタフェースをどう設計するかは、別々ではなく一緒に考えるべきだと報告しています [28]。これは、第 2 章の Blackboard や第 4 章の PRS で扱ってきた「知識源をどうつなぐか」という問いが、ロボットの文脈でもう一度問われているとも読めます。

層	主な役割	最近の例	古典研究との対応
推論・計画層	タスク分解、進捗の推定、再計画、ツール呼び出し	Gemini Robotics-ER、LLM Planner	BDI の Intention、PRS の Interpreter
スキル層	実際の動作の生成、動作プリミティブ	VLA Policy、Code as Policies	PRS の Plan Library、Knowledge Source
記憶・状態層	シーン、タスクの状態、失敗の履歴を保持する	Scene Graph、Task Memory	Belief Database、Blackboard
検証・回復層	成功の判定、異常の検知、やり直し	Success Detector、進捗推定	BDI の Reconsideration

12.3 経験から学び続けるロボット

Agentic Robotics のもう一つの流れは、ロボットが実際に動いた経験から学び続けることです。Physical Intelligence の π*0.6 では、RECAP と呼ばれる方法で、デモンストレーション、ロボット自身の試行のデータ、自律動作中に人間が行った修正をまとめて強化学習に使っています。家庭での洗濯物たたみ、箱の組み立て、業務用マシンでのエスプレッソづくりなどに取り組み、難しいタスクでは処理量が 2 倍以上に、失敗率がおよそ半分になったと報告されています [27]。

知識科学の言葉で言いかえると、これは「行動によって知識を確かめ、更新する」サイクルを、ロボットの身体のレベルで回していることになります。このサイクルをエージェント自身の改善にまで広げた考え方が、次の章で扱う RSI です。

12.4 知識科学から見た論点

1. 推論と運動制御の境界（agent–policy seam）は、どこに置くのがよいのでしょうか。いまの分け方が長く使えるものなのか、一時的なものなのかも議論されています [29]。
2. 人間からのフィードバックは、どの抽象度に戻すのがよいのでしょうか。手順の誤りは言葉で、動作の誤りは身体的な教示で直す、といった振り分けが提案されています [29]。
3. 失敗から立ち直る力（Recovery）を、ベンチマークのごまかしを招かずにどう評価すればよいのでしょうか [29]。
4. 各層が持っている知識を Ontology や Knowledge Graph とどう対応づければ、説明や検証がしやすくなるのでしょうか。

第 13 章　再帰的自己改善（RSI）

13.1 RSI とは

再帰的自己改善（Recursive Self-Improvement, RSI）とは、AI システムが自分自身を改善し、その結果として「改善する力」そのものも高まっていく、という考え方です。古くは Good が 1965 年に、自分より優れた機械を設計できる機械が生まれれば、知能が爆発的に高まりうると論じました [30]。Schmidhuber のゲーデルマシンは、自分の書き換えが有益だと証明できたときにだけ自分を書き換える、という理論的な自己改善システムを提案しています [31]。

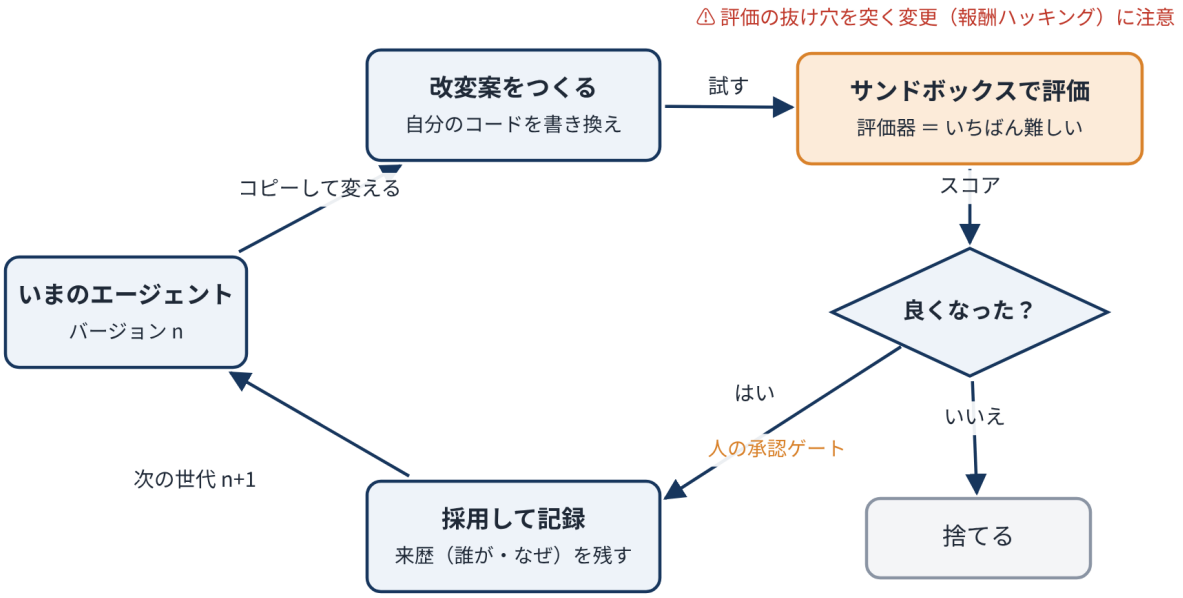


図 14　自己改善のループ。オレンジの「評価」が弱いと、見かけの数字だけ良い変更が採用されてしまいます。

ただ、いま実際に行われている研究の多くは、証明ではなく実験的な評価にもとづいて「良くなった変更だけを残す」タイプのものです。改善の対象も、モデル全体ではなく、プロンプトやコード、スキルなどの一部に限られていることがほとんどです。「再帰的」と言っても、どこまで本当に再帰しているのかは、論文ごとに丁寧に見ていく必要があります。

13.2 何を改善するのか

2026 年のサーベイでは、自己改善を「何を更新するか」で大きく二つに分けています。モデルのパラメータを更新する方法と、プロンプト、記憶、ツール、制御ロジックといったモデルの周りの足場（Scaffolding）を更新する方法です。前者は時間がかかるものの効果が広く長続きし、後者は速く、元に戻しやすいという特徴があります [37]。

改善の対象	代表的な研究	内容
スキル・知識	Voyager [32]	Minecraft の中で身につけたスキルをコードとしてライブラリに蓄え、再利用する

改善の対象	代表的な研究	内容
改善プログラム自体	STOP [33]	コードを改善するプログラムが、自分自身のコードも改善する
エージェントのコード	Darwin Gödel Machine [35]、AIDE² [38]	エージェントが自分のコードを書き換え、評価で良くなったものだけを残す
アルゴリズム・計算基盤	AlphaEvolve [34]	進化的な探索でアルゴリズムを改良し、自身を支える LLM の学習も高速化した
ロボットの方策	RoboCat [42]、Self-Improving EFM [36]、π*0.6 [27]	実機での練習や人の修正データから、ロボット自身の方策を改善する

13.3 代表的な研究

Darwin Gödel Machine は、コーディングエージェントが自分のコードを書き換え、そのたびにベンチマークで評価して、良い変種をアーカイブに残していく仕組みです。SWE-bench では 20.0%から 50.0%へ、Polyglot では 14.2%から 30.7%へと性能が上がりました。実験はサンドボックスと人間の監督のもとで行われています [35]。

AlphaEvolve は、LLM を使ってアルゴリズムのコードを書き換え、評価器からのフィードバックを手がかりに進化的に改良していくエージェントです。データセンターのスケジューリングやハードウェア回路の改善に加えて、AlphaEvolve 自身を支える LLM の学習を高速化したと報告されています。4×4 の複素行列の積を 48 回のスカラー乗算で計算する方法も見つけています [34]。

2026 年の AIDE²では、AI 研究エージェントのコードを繰り返し書き換え、評価で良くなった場合だけ採用する二重ループの仕組みを試しています。8 日間の自律的な実行で 7 つの改善が採用され、その効果は外部の 4 つのベンチマークにも引き継がれました。一方で、主な結果が 1 回の実行にもとづくこと、ほかの実行では採用された改善が 2 つや 4 つにとどまったこと、「より良い自己改善者になったか」の検証はまだ決着していないことなど、限界も率直に書かれています [38]。

13.4 ロボティクスにおける RSI

ロボットの世界でも、自己改善の研究が進んでいます。RoboCat は、新しいタスクのデモから学んだあと、自分で集めたデータを使って次の世代のモデルを訓練する、という自己改善のサイクルを示しました [42]。Ghasemipour らは、まず模倣学習と「ゴールまであと何ステップか」の予測でモデルを調整し、次にその予測を報酬関数と成功判定器として使うことで、複数のロボットが人の手をあまり借りずに練習できるようにしました。その結果、デモを増やすよりも効率よく性能を伸ばし、デモにない新しいスキルも身につけられたと報告しています [36]。

ただし、物理世界にはデジタル世界にはない制約があります。行動は取り消せないことが多く、試行のたびに環境をリセットする手間がかかり、ハードウェアも摩耗します。安全に試せる範囲をどう決めるか、シミュレーションをどう活用するかが、ロボットの RSI ではとくに大事なテーマになります [29]。

13.5 知識科学から見た RSI

本稿の視点から見ると、RSI は「知識の獲得 → 形式化 → 検証 → 更新」という循環を、エージェント自身に向けて自動で回す試みだと捉えられます。そう考えると、RSI のボトルネックは改善案を生み出すことよりも、「その改善が本当に良いものかを確かめる評価器」の側にあることが見えてきます。

論点	内容	関連する章
評価器への依存	評価が不完全だと、評価の数字だけを上げる変更（報酬ハッキング）が選ばれてしまう	第 5 章（入札の検証）
目標のずれ	改善を重ねるうちに、もともとの目的や制約から少しずつ離れていくおそれがある	第 3 章（BDI のコミットメント）
来歴と監査	誰が・いつ・なぜ変更したかを記録しておかないと、あとから確かめられない	第 1 章・第 6 章（知識の来歴）
人間の承認	どの変更を自動で採用し、どれを人間が確認するかの線引きが必要になる	第 14 章（安全性）

つまり、RSI を安全に進めるためには、変更の来歴を残すこと、サンドボックスで試すこと、人間が承認するポイントをあらかじめ決めておくことが欠かせません。これは、Ontology や Knowledge Graph で知識の来歴や有効期間を管理するという、知識工学が積み重ねてきた考え方とそのまま重なっています。

第 14 章　本稿でまだ足りていない視点

14.1 この章を書いた理由

ここまで、「知識の扱い方」という一本の軸で AI エージェントの歴史をたどってきました。軸を一本にしたぶん見通しはよくなりましたが、そのぶん抜け落ちている視点もあります。この章では、今後の版で補いたい論点を正直に書き出しておきます。ゼミでの議論のきっかけになればうれしいです。

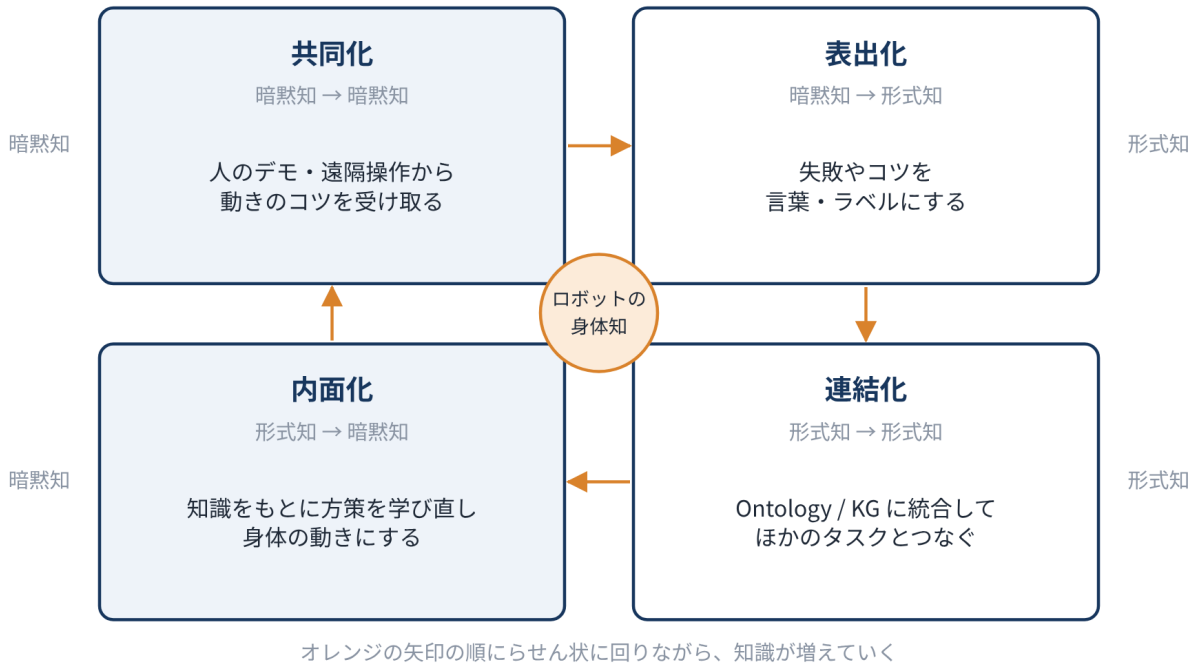


図 15　SECI モデル（野中・竹内）をロボットに当てはめた図。第 14 章で「足りない視点」として挙げた、知識が生まれる過程です。

視点	いまの本稿で足りない点	補うとよい論点
身体性と行動ベース AI	記号から学習へという流れが中心で、記号を使わない知能の系譜が抜けている	Brooks の Subsumption Architecture [5]、記号接地問題 [43]
知識創造（SECI）	知識を「処理する」視点が中心で、知識が「生まれる」過程を扱えていない	野中・竹内の SECI モデル [39]、暗黙知と形式知の変換
不確実性と確率的推論	確率的な信念の扱いがほとんど出てこない	POMDP、ベイズ推論、確率ロボティクス
人間との協働	Agent 同士の協調が中心で、人間との関係が薄い	Human-in-the-loop、信頼、説明、役割分担、HRI
安全性とガバナンス	安全性についての議論がほとんどない	報酬ハッキング、取り消せない行動、権限の管理 [44]
通信プロトコルと相互運用	FIPA ACL のあとの流れが書けていない	MCP [40]、A2A [41] と Ontology の関係
評価とベンチマーク	技術どうしを比べる基準が示せていない	長期タスクの評価、実機での評価、再現性

視点	いまの本稿で足りない点	補うとよい論点
記憶・忘却・継続学習	Memory を「ためる」側だけで、「忘れる・整理する」側がない	破滅的忘却、記憶の統合、知識の陳腐化
計算資源と実時間性	計算コストや遅延について考えていない	エッジでの推論、エネルギー、制御周期
社会・倫理・法	社会のなかでの位置づけに触れていない	責任の所在、プライバシー、仕事への影響

14.2 とくに補いたい視点 1：身体性と記号接地

Brooks は、記号的な世界モデルを持たなくても、感覚と行動を直接つなぐ層を積み重ねることで、ロボットが頑健に動けることを示しました [5]。また Harnad は、記号の意味がほかの記号によってしか定義されないのであれば、記号はいつまでたっても世界に結びつかない、という記号接地問題を指摘しました [43]。LLM や VLA の時代になって、この問いはむしろ現実の問題として戻ってきています。知能ロボティクスの立場からは、第 11 章の Grounding の議論を、この系譜と結びつけて書き直したいと考えています。

14.3 とくに補いたい視点 2：SECI モデルとロボットの身体知

JAIST の知識科学にとって、野中・竹内の SECI モデルは欠かせない枠組みです [39]。SECI モデルでは、暗黙知と形式知が、共同化（Socialization）、表出化（Externalization）、連結化（Combination）、内面化（Internalization）の四つのプロセスを通じて変換され、新しい知識が生まれると考えます。これをロボットの学習に当てはめてみると、次のように対応づけられそうです。

SECI のプロセス	ロボット学習での対応例
共同化（暗黙知→暗黙知）	人間のデモや遠隔操作を通じて、言葉にしにくい動きのコツをロボットが受け取る
表出化（暗黙知→形式知）	失敗の原因や動作のコツを、言葉・ラベル・Failure Concept として書き表す
連結化（形式知→形式知）	書き表した知識を Ontology や Knowledge Graph に統合し、ほかのタスクの知識と結びつける
内面化（形式知→暗黙知）	統合された知識をもとに方策を学び直し、ふたたび身体の動きとして身につける

こうして並べてみると、第 12 章・第 13 章で見た「経験から学び続けるロボット」は、SECI のらせんを機械の上で回そうとしている試みとも読めます。知識科学と知能ロボティクスをつなぐ切り口として、今後もう少し深めていきたいところです。

14.4 とくに補いたい視点 3：安全性と人間との協働

エージェントが道具を使い、物理世界で動き、さらに自分自身を改善するようになると、安全性の問題は避けて通れません。報酬関数の抜け穴を突く行動、取り消せない副作用、想定していない環境での失敗などは、以前から具体的な課題として整理されてきました [44]。本稿では、こうしたリスクや、人間がどこで確認・承認を行うのかという協働の設計について、まだほとんど書けて

いません。次の版では、技術の歴史と並べて、安全性と人間との関係の歴史も整理してみたいと考えています。

第 15 章　歴史の総括と研究課題

15.1 技術史を知識の扱い方から整理する

時代	代表技術	中心となる問い	知識科学との関係
1950 から 70 年代	記号主義 AI、Expert System	知識をどう表現し推論するか	知識表現、知識獲得
1970 から 80 年代	HEARSAY-II、Blackboard	異なる知識源をどう協調させるか	知識統合、共有
1980 から 90 年代	BDI、PRS	知識と目標からどう行動するか	実践的推論、手続き的知識
1980 から 2000 年代	Contract Net、FIPA ACL	複数主体がどう交渉し協調するか	分散知識、社会的協調
1990 から 2000 年代	Ontology、Semantic Web	異なる System で意味をどう共有するか	概念化、相互運用
2010 年代	Deep Learning、RL	経験から知識と行動をどう学ぶか	明示知と潜在知
2022 年以降	ReAct、LLM Agent	自然言語でどう計画し行動するか	知識獲得の自動化
2023 年以降	LLM Multi-Agent	複数 LLM をどう協調させるか	知識共有、役割分担
2024 年以降	GraphRAG、LLM KG	外部知識をどう構造化して使うか	知識表現、検証
2025 年以降	Agentic Robotics、階層型 VLA	推論とスキルを実機でどう組み合わせるか	身体化された計画、検証
2025 年以降	Self-Improving Agent、RSI	エージェントが自分自身をどう改善するか	知識循環の自動化、来歴管理
現在	VLA、World Model	知識を物理行動へどう結びつけるか	身体化、相互作用

それぞれの技術は、前の技術を完全に置き換えたわけではありません。Blackboard、BDI、Ontology は、いまの AI システムにも組み込める設計原理として生き続けています。AI エージェントの歴史は、古いやり方が消えていく歴史というより、さまざまな知識処理の方法が積み重なり、組み合わせ直されてきた歴史として理解できるのではないでしょうか。

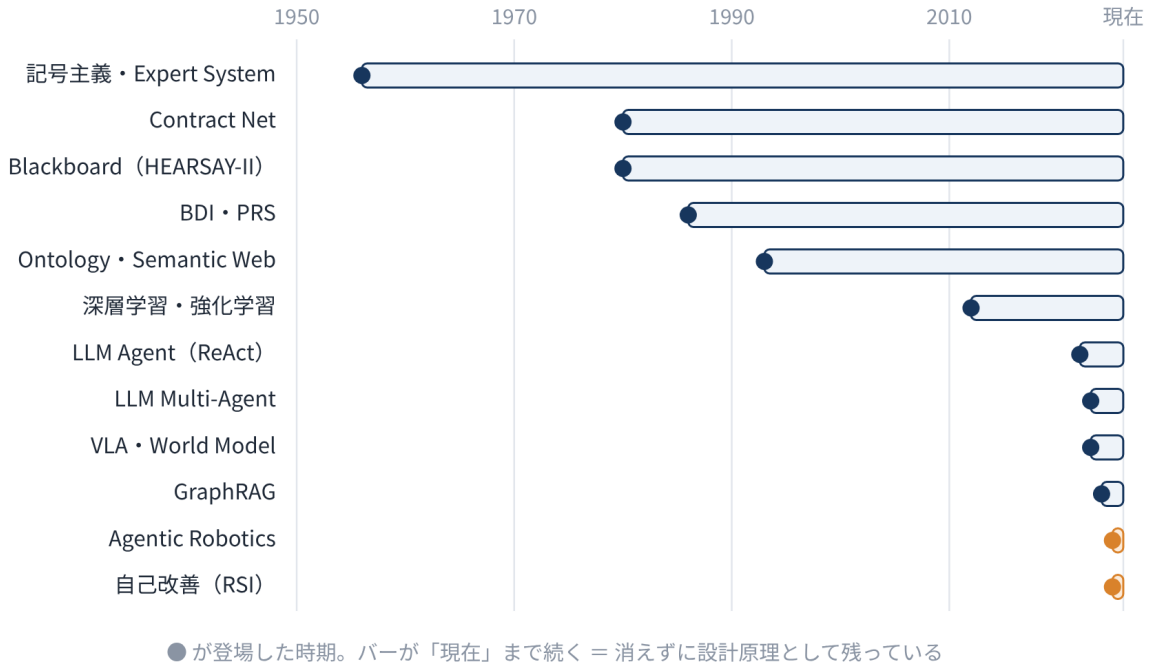


図 16　技術は「置き換え」ではなく「積み重なり」。どのバーも現在まで伸びていて、いまの AI エージェントの部品として使われています。

15.2 ゼミで提示したい Research Question

RQ	研究質問	主な評価観点
RQ1 知識獲得	LLM を利用し、自然言語や観測 Data から正確で検証可能な知識を継続的に獲得できるか	正確性、完全性、出典、Human Review 工数
RQ2 内部構造	BDI の Belief Goal Intention を外部 Memory と統合し、長期行動の一貫性を高められるか	完遂率、再計画、矛盾、再開可能性
RQ3 協調	Contract Net、Blackboard、共通 Ontology により Agent 間協調を改善できるか	重複、割当精度、Cost、合意品質
RQ4 動的知識	Temporal KG で知識の時間的変化を管理し、現在有効な情報に基づいて意思決定できるか	時間整合性、陳腐化、来歴、監査性
RQ5 Physical AI	Ontology や KG と World Model を統合し、Robot Action を説明可能で検証可能にできるか	成功率、失敗予測、制約違反、説明性
RQ6 Agentic Robotics	推論層と制御層の境界をどこに置き、人間のフィードバックをどの抽象度に戻すと、長期タスクの成功と失敗からの回復が改善するか	長期タスク成功率、回復率、介入回数、汎化
RQ7 自己改善	ロボットが実世界の経験から、安全制約を守りながらスキルと知識を改善し続けられるか	改善率、人手介入量、報酬ハッキング、制約違反、来歴

15.3 本稿の結論

初期の AI は、人間が与えた知識を使って推論するシステムでした。BDI と PRS によって、知識を持つだけでなく、目標を選び、計画を保ち、環境に合わせて行動する Agent が研究されるようになりました。Contract Net と Blackboard は複数の知識源や Agent が協調する仕組みを、Ontology は異なる System のあいだで知識の意味を共有する基盤を用意しました。

LLM は、自然言語から知識の候補や行動計画を生み出し、知識獲得と形式化の前提を大きく変えました。けれども、自然言語を扱えることと、知識を正しく理解して一貫した意思決定ができることは同じではありません。BDI の内部状態、Contract Net の協調、Ontology の意味論、Knowledge Graph の明示的な表現は、いまの LLM Agent にとっても大切な研究対象です。

Physical AI や Agentic Robotics では、知識を言語や Graph のなかに持っておくだけでなく、現実世界での認識・予測・行動へと結びつける必要があります。さらに RSI の研究は、エージェントが自分自身の手順やスキルまで更新していく可能性を示しています。これからの中心的な課題は、知識を記述する AI から、知識を獲得し、共有し、行動によって確かめながら更新していく AI への移行だと考えています。知識工学と知識科学は、その設計と評価に欠かせない視点を与えてくれるはずです。

参考文献

[1] Newell, A. (1982). The Knowledge Level. Artificial Intelligence, 18(1), 87-127. Link

[2] Erman, L. D., Hayes-Roth, F., Lesser, V. R., & Reddy, D. R. (1980). The HEARSAY-II Speech-Understanding System. ACM Computing Surveys, 12(2), 213-253. Link

[3] Smith, R. G. (1980). The Contract Net Protocol. IEEE Transactions on Computers, C-29(12), 1104-1113. Link

[4] Georgeff, M. P., & Lansky, A. L. (1986). Procedural Knowledge. Proceedings of the IEEE, 74(10), 1383-1398. Link

[5] Brooks, R. A. (1986). A Robust Layered Control System for a Mobile Robot. IEEE Journal on Robotics and Automation, 2(1), 14-23. Link

[6] Bratman, M. E. (1987). Intention, Plans, and Practical Reason. Harvard University Press.

[7] Gruber, T. R. (1993). A Translation Approach to Portable Ontology Specifications. Knowledge Acquisition, 5(2), 199-220. Link

[8] Rao, A. S., & Georgeff, M. P. (1995). BDI Agents: From Theory to Practice. Proceedings of ICMAS. Link

[9] Rao, A. S. (1996). AgentSpeak(L): BDI Agents Speak Out in a Logical Computable Language. MAAMAW. Link

[10] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. Scientific American, 284(5), 34-43. Link

[11] W3C. (2004). OWL Web Ontology Language Overview. Link

[12] Mnih, V., et al. (2015). Human-level Control through Deep Reinforcement Learning. Nature, 518, 529-533. Link

[13] Silver, D., et al. (2016). Mastering the Game of Go with Deep Neural Networks and Tree Search. Nature, 529, 484-489. Link

[14] Ahn, M., et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. Link

[15] Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. Link

[16] Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. Link

[17] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. Link

[18] Babaei Giglou, H., D'Souza, J., & Auer, S. (2023). LLMs4OL: Large Language Models for Ontology Learning. Link

[19] Brohan, A., et al. (2023). RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. Link

[20] Edge, D., et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Link

[21] Kim, M. J., et al. (2024). OpenVLA: An Open-Source Vision-Language-Action Model. Link

[22] Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. Link

[23] Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL, 12, 157-173. Link

[24] Black, K., et al. (2024). pi0: A Vision-Language-Action Flow Model for General Robot Control. Link

[25] Liang, J., et al. (2023). Code as Policies: Language Model Programs for Embodied Control. ICRA 2023. Link

[26] Gemini Robotics Team. (2025). Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer. Link

[27] Physical Intelligence. (2025). π*0.6: a VLA That Learns From Experience. Link

[28] Hu, J., et al. (2026). What Matters in Orchestrating Robot Policies: A Systematic Study of Hierarchical VLA Agents. Link

[29] Workshop on Agentic Robotics. (2026). CoRL 2026 Workshop. Link

[30] Good, I. J. (1965). Speculations Concerning the First Ultraintelligent Machine. Advances in Computers, 6, 31-88. Link

[31] Schmidhuber, J. (2003). Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. Link

[32] Wang, G., et al. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. Link

[33] Zelikman, E., et al. (2024). Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. COLM 2024. Link

[34] Novikov, A., et al. (2025). AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. Link

[35] Zhang, J., Hu, S., Lu, C., Lange, R., & Clune, J. (2026). Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. ICLR 2026. Link

[36] Ghasemipour, S. K. S., et al. (2025). Self-Improving Embodied Foundation Models. NeurIPS 2025. Link

[37] Ren, Z., et al. (2026). Self-Improvements in Modern Agentic Systems: A Survey. Link

[38] Srikanth, D., et al. (2026). Recursive Self-Improvement of AI Research Agents. Link

[39] Nonaka, I., & Takeuchi, H. (1995). The Knowledge-Creating Company. Oxford University Press.

[40] Anthropic. (2024). Introducing the Model Context Protocol. Link

[41] Google. (2025). Announcing the Agent2Agent Protocol (A2A). Link

[42] Bousmalis, K., et al. (2023). RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation. Link

[43] Harnad, S. (1990). The Symbol Grounding Problem. Physica D, 42, 335-346. Link

[44] Amodei, D., et al. (2016). Concrete Problems in AI Safety. Link